Generative Artificial Intelligence

Assignment #2 Report

Denoising Autoencoder and CycleGAN for Face–Sketch Translation

Name: Hamas Naveed

Roll No: 23i-3014

Name: Azfar

Roll No: 23i-3055

Semester 7

Code: https://github.com/HamasNaveed/Gen-AI-Assignment2

Question 1: Image Denoising with a Denoising Autoencoder

1.1 Dataset preparation

The Berkeley Segmentation Dataset (BSDS300) was downloaded programmatically. It contains 200 training and 100 test images. All images were resized to 128×128 RGB and normalised to the range [0, 1]. The 200 training images were randomly split (fixed seed 42) into 160 for training and 40 for validation; the 100 official test images form the test set. This gives a 160 / 40 / 100 split.

1.2 Noise injection

Two corruptions were applied to the clean images, always keeping the clean image as the training target:

- Gaussian noise: $x' = \text{clip}(x + N(0, \sigma^2), 0, 1)$, with $\sigma \in \{0.05, 0.1, 0.2, 0.3\}$.
- Salt-and-pepper noise: a fraction p of pixel locations is set to 0 or 1 with equal probability, $p \in \{0.05, 0.1, 0.2, 0.3\}$.

Noise is regenerated for every batch during training, so the network never sees the same corrupted image twice, which acts as data augmentation.

1.3 Model architecture

The model is a fully convolutional denoising autoencoder with about 1.09 million parameters.

- Encoder: four 3×3 convolutions with stride 2 (3→32→64→128→256 channels), each followed by BatchNorm and ReLU, reducing 128×128 inputs to 8×8 feature maps. A final 1×1 convolution projects to the bottleneck.
- Bottleneck: 16 channels × 8 × 8 = 1,024 values by default (about 2% of the 49,152 input values). The bottleneck size is varied in the experimental study.
- Decoder: a 1×1 convolution with ReLU, then three 4×4 stride-2 transposed convolutions (with BatchNorm and ReLU) and a final transposed convolution to 3 channels with a Sigmoid activation, so outputs lie in [0, 1].
- Activations: ReLU in hidden layers; Sigmoid at the output.

1.4 Training

The network was trained to map noisy images to the clean originals using the mean squared error (MSE) reconstruction loss. Optimiser: Adam, learning rate 1e-3 halved on validation plateau (patience 5), batch size 16, 60 epochs for the main models (30 epochs for each model in the experimental study). The checkpoint with the lowest validation loss was kept. Figure 1 shows the training and validation curves for both noise types at level 0.1; both decrease steadily and the validation loss closely follows the training loss, indicating little overfitting.

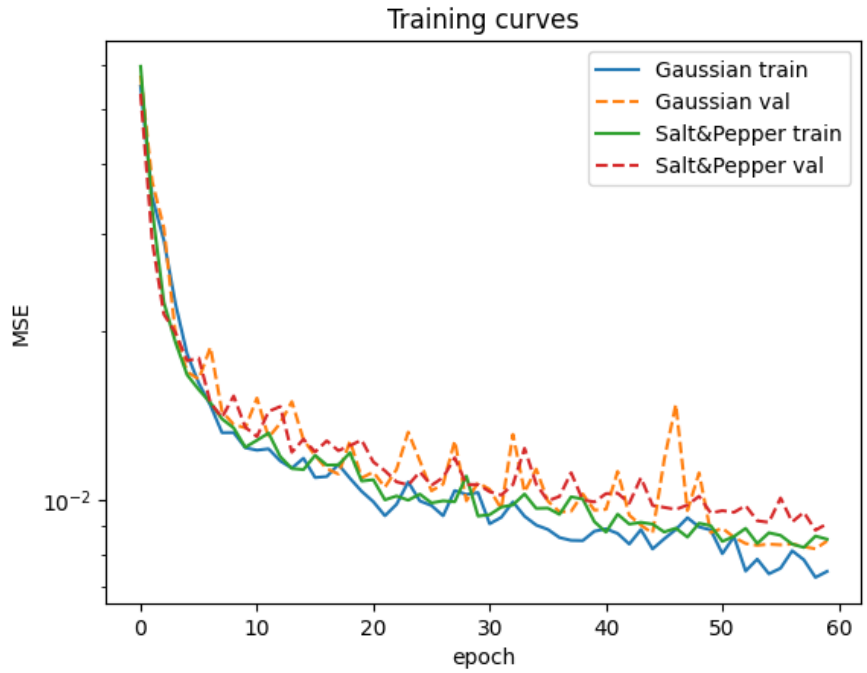


Figure 1: Training and validation MSE (log scale), noise level 0.1.

1.5 Evaluation and visualisation

Models were evaluated on the 100 test images using MSE, PSNR (dB) and SSIM, and compared with the noisy input as a baseline. At noise level 0.1:

Noise type	MSE	PSNR (dB)	SSIM	Noisy PSNR	Noisy SSIM
Gaussian	0.0071	22.08	0.557	20.32	0.476
Salt & pepper	0.0078	21.65	0.510	15.05	0.310

Table 1: Test-set results at noise level 0.1 (60-epoch models).

Both models improve on the noisy input. The gain is much larger for salt-and-pepper noise (+6.6 dB) than for Gaussian noise (+1.8 dB), because impulse noise produces extreme outliers that the network learns to detect and replace. Figures 2 and 3 show original, noisy and denoised images. Gaussian grain is largely removed, while some fine texture is smoothed away; salt-and-pepper specks are removed but the output is slightly blurry.

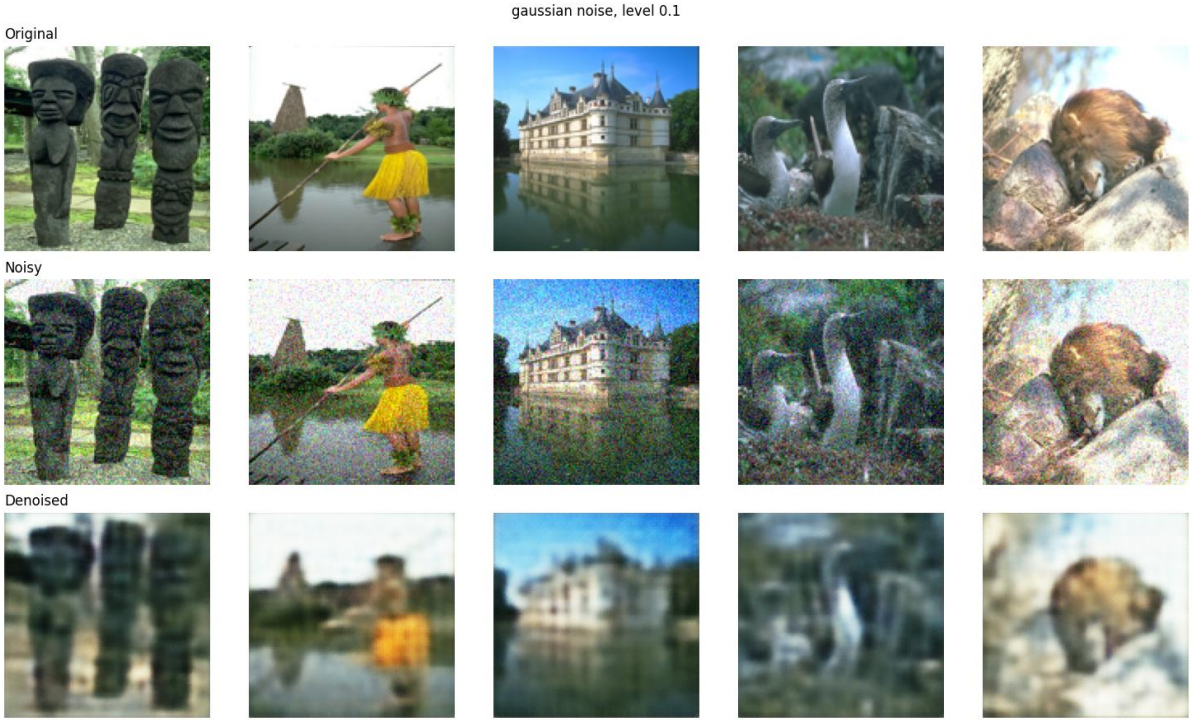


Figure 2: Original (top), Gaussian-noisy σ = 0.1 (middle) and denoised (bottom).

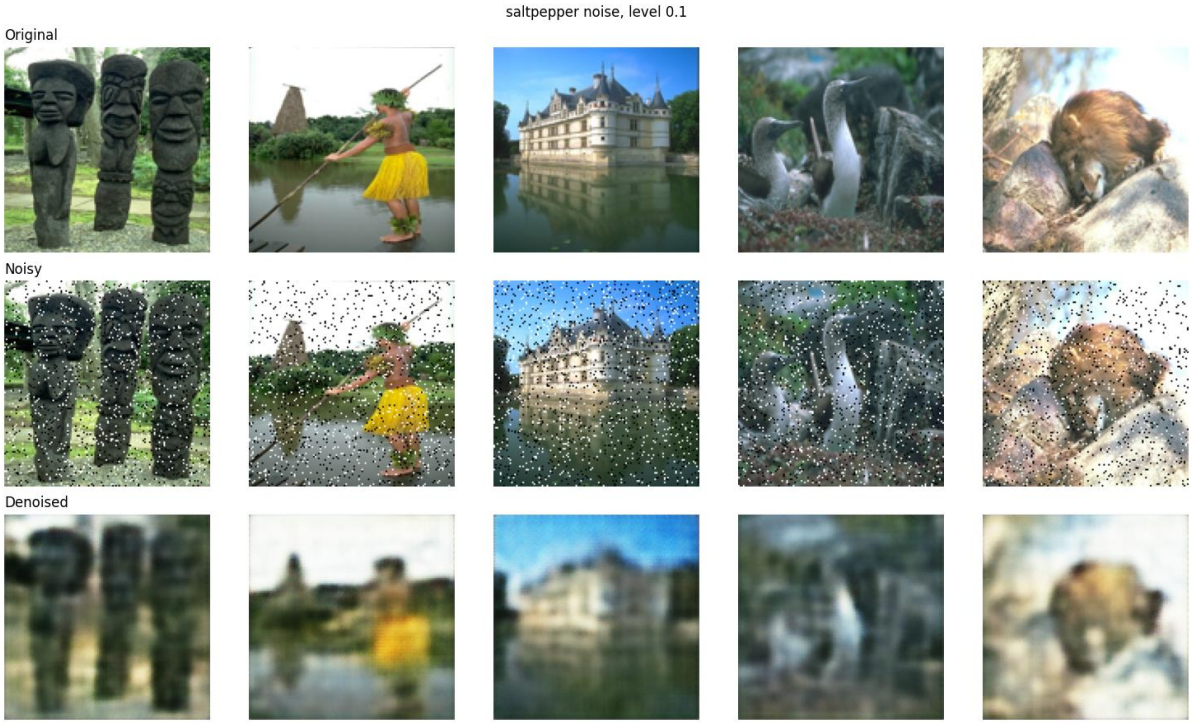


Figure 3: Original (top), salt-and-pepper p = 0.1 (middle) and denoised (bottom).

1.6 Experimental study

(a) Effect of noise level

A separate model (bottleneck 16 channels) was trained for each noise type and level and evaluated on the test set.

Noise type	Level	MSE	PSNR (dB)	SSIM	Noisy-input PSNR	Noisy-input SSIM
Gaussian	0.05	0.0092	20.94	0.480	26.18	0.710
Gaussian	0.10	0.0087	21.19	0.479	20.32	0.476
Gaussian	0.20	0.0088	21.08	0.473	14.80	0.258
Gaussian	0.30	0.0093	20.82	0.460	11.95	0.164
Salt & pepper	0.05	0.0090	21.01	0.471	18.05	0.482
Salt & pepper	0.10	0.0091	20.91	0.469	15.05	0.310
Salt & pepper	0.20	0.0095	20.72	0.452	12.05	0.179
Salt & pepper	0.30	0.0101	20.45	0.432	10.27	0.120

Table 2: Reconstruction quality versus noise level.

(b) Effect of bottleneck size

Noise level was fixed at 0.1 and the number of bottleneck channels varied (latent size = 64 × channels).

Channels	Latent size	G: MSE	G: PSNR	G: SSIM	S&P: MSE	S&P: PSNR	S&P: SSIM
2	128	0.0131	19.46	0.364	0.0134	19.32	0.362
4	256	0.0104	20.43	0.429	0.0108	20.23	0.424
8	512	0.0090	20.99	0.479	0.0094	20.78	0.460
16	1024	0.0090	20.98	0.480	0.0091	20.93	0.469
32	2048	0.0085	21.25	0.492	0.0090	20.99	0.473
64	4096	0.0087	21.13	0.485	0.0092	20.91	0.466

Table 3: Reconstruction quality versus bottleneck size (G = Gaussian, S&P = salt-and-pepper).

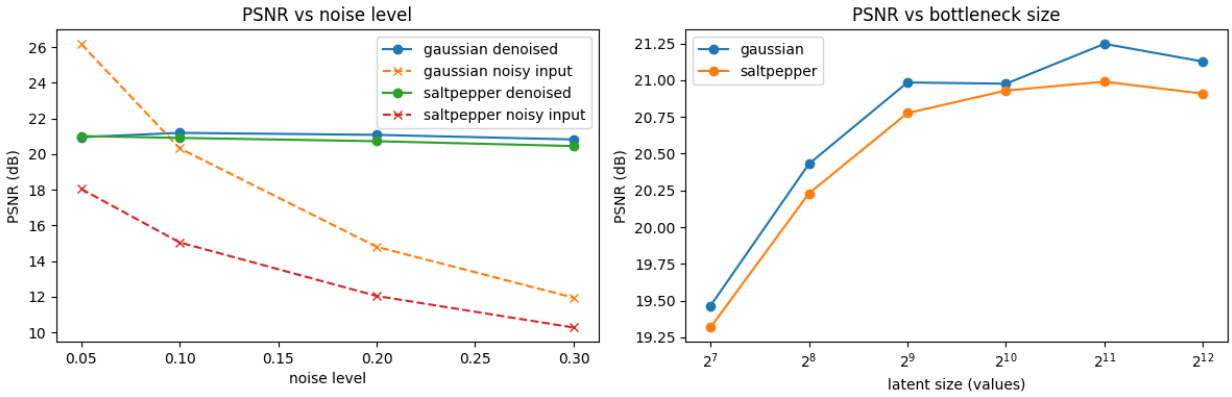


Figure 4: PSNR versus noise level (left) and versus latent size (right).

Observations

- The denoised PSNR is almost flat (20.4–21.2 dB) across noise levels, whereas the noisy-input PSNR falls sharply (Gaussian: 26.2 → 11.9 dB). The benefit of the autoencoder therefore grows with the noise level, e.g. +8.9 dB at Gaussian σ = 0.3 and +10.2 dB at salt-and-pepper p = 0.3.
- At very low Gaussian noise (σ = 0.05) the autoencoder is worse than leaving the image untouched (20.9 vs 26.2 dB): the 8×8 bottleneck loses fine detail that the almost-clean input already contained. Reconstruction error is dominated by this information loss rather than the noise.
- Salt-and-pepper noise is harder at the highest level: performance drops from 21.0 to 20.4 dB, faster than Gaussian (21.2 to 20.8 dB), since corrupted pixels carry no information about their true value.
- Bottleneck size matters at the small end: shrinking the latent from 512 to 128 values lowers PSNR by about 1.5 dB and SSIM from 0.48 to 0.36, because the code cannot hold enough image detail. Beyond roughly 512–2048 values, gains saturate (best: 2048 values, 21.25 dB), and larger codes (4096) bring no benefit.
- Limitations: all models are small and trained on only 160 images with 30 epochs in the study, so differences below about 0.3 dB between settings should not be over-interpreted (single run, no repeated seeds).

Question 2: CycleGAN for Person Face Sketches

2.1 Dataset and preprocessing

The Person Face Sketches dataset (Kaggle, almightyj/person-face-sketches) contains 22,334 photographs and 22,334 sketches, organised into train / val / test folders. The two domains are unpaired from the model's point of view: for each real face a random sketch is drawn. Because of time limits on Colab, 500 images per domain were used (the reduction of samples allowed by the assignment), split 90% / 10% for training and held-out testing. Training images are resized to 286 pixels, randomly cropped to 128×128 and randomly flipped horizontally; test and UI images are simply resized to 128×128. All images are normalised to [−1, 1].

2.2 Model

The implementation follows Zhu et al. (ICCV 2017):

- Two generators, G_AB (face → sketch) and G_BA (sketch → face): 7×7 conv, two stride-2 downsampling convs, 6 residual blocks (reduced from 9 because of the smaller 128×128 resolution), two transposed-conv upsampling layers, 7×7 output conv with Tanh. Instance normalisation and reflection padding are used throughout.
- Two PatchGAN discriminators D_A and D_B (70×70 receptive field): four 4×4 convs (64–128–256–512 filters) with LeakyReLU(0.2) and instance norm, ending in a 1-channel patch score map.
- Losses: least-squares GAN adversarial loss, cycle-consistency L1 loss (λ = 10) in both directions, and identity L1 loss (λ = 5). A history pool of 50 generated images is used when updating the discriminators.
- Weights are initialised from N(0, 0.02).

2.3 Training setup

Setting	Value
Hardware	Google Colab GPU runtime
Training images	500 faces + 500 sketches (subset)
Image size	128 × 128
Epochs	20 (about 82 s per epoch)
Batch size	4
Optimiser	Adam, lr 2e-4, β = (0.5, 0.999)
LR schedule	constant for 10 epochs, then linear decay to 0
Loss weights	λ_cycle = 10, λ_identity = 5

Table 4: Training configuration.

Checkpointing: after every epoch the full training state (both generators and discriminators, optimisers, LR schedulers, loss history, epoch number) is written to Google Drive via an atomic save, together with standalone generator weights. When the Colab session was interrupted, re-running the notebook

automatically resumed from the last saved epoch (training was resumed from epoch 5 in the final run), exactly as required by the assignment.

2.4 Results

Figure 5 shows the losses during training. All loss terms decrease steadily; the discriminator loss stays below the generator's adversarial pressure but does not collapse to zero, which indicates that the adversarial game remains balanced.

Epoch	Generator total	Discriminator	Cycle loss	Identity loss
6	4.758	0.464	0.273	0.238
10	4.545	0.364	0.243	0.204
15	4.146	0.280	0.220	0.186
20	3.821	0.235	0.197	0.167

Table 5: Mean training losses at selected epochs.

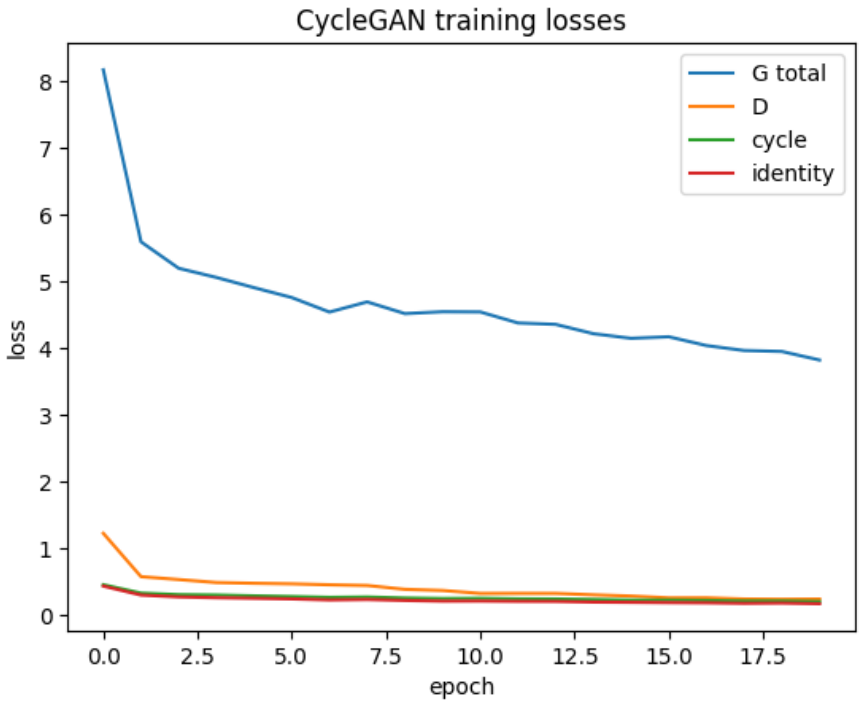


Figure 5: CycleGAN training losses.

Figure 6 shows results on held-out test images after 20 epochs. Each triple shows the input, the translation and the cycle reconstruction. Rows: real face, face → sketch, reconstructed face; sketch, sketch → face, reconstructed sketch.

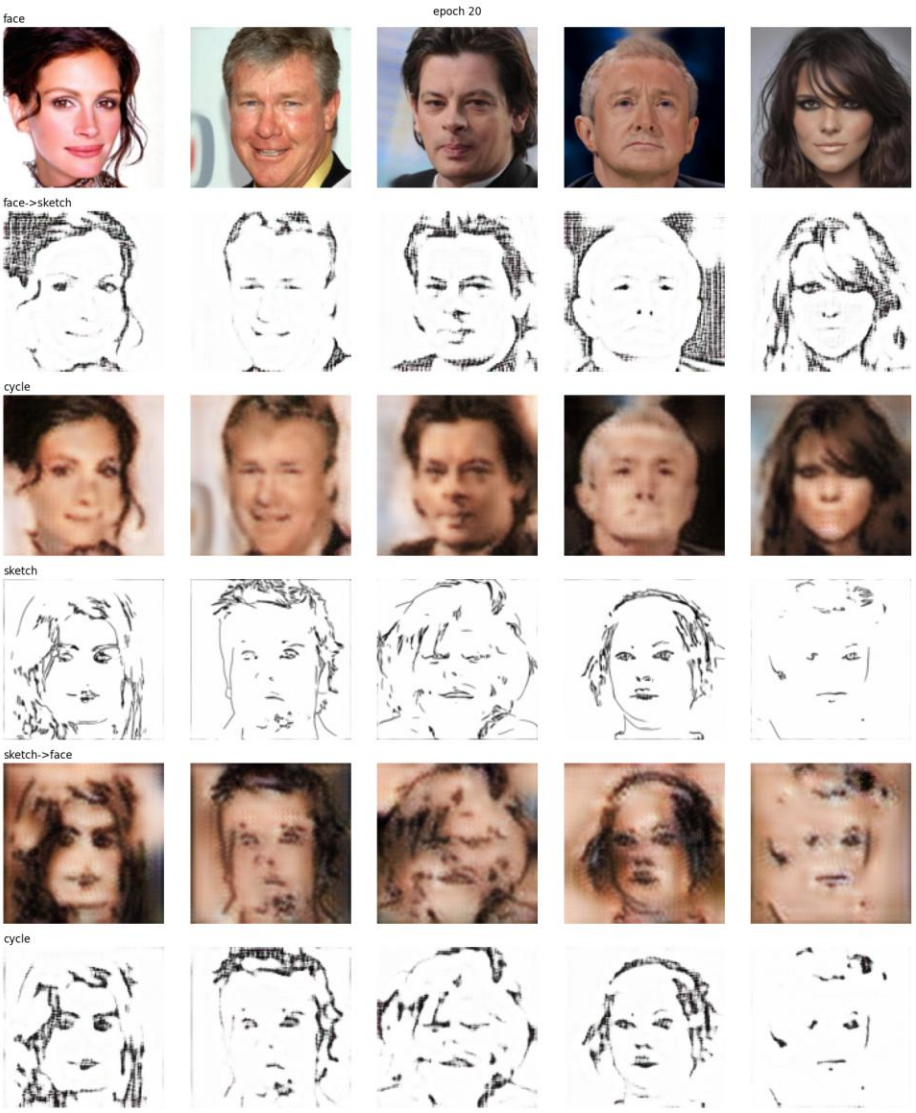


Figure 6: Test results after 20 epochs.

Early in training (epoch 6, Figure 7) outputs are blurry with weak structure; by epoch 20 the face → sketch outputs show clear outlines of hair, eyes, nose and mouth on a white background.

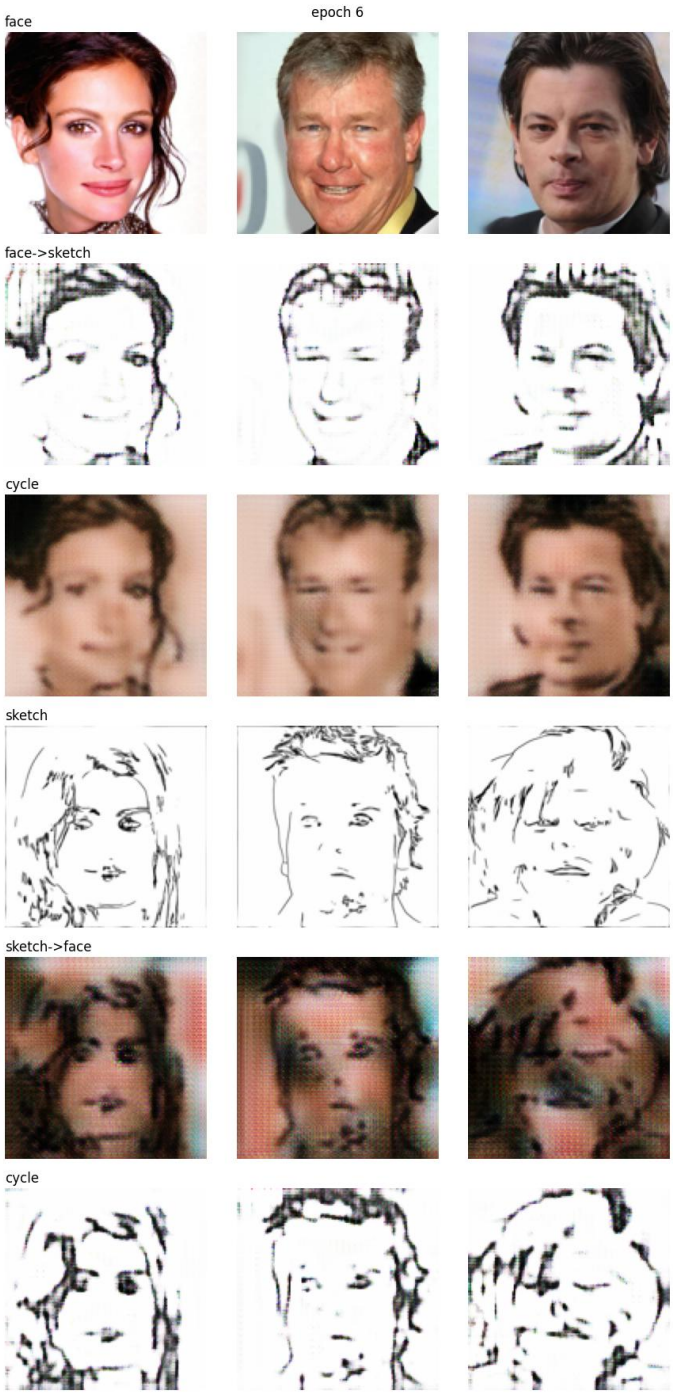


Figure 7: Results at epoch 6 for comparison.

Observations and limitations

- Face → sketch is the stronger direction: facial structure and hair edges are preserved, though a cross-hatched texture artefact appears in dark regions such as hair.
- Sketch → face is clearly weaker. The generator has to invent colour and texture from sparse line art, so the outputs are blotchy and sometimes distorted, and dark strokes leak into the skin

tone. The cycle reconstructions are nevertheless close to the original inputs, which shows the cycle-consistency constraint is being satisfied (cycle loss 0.27 → 0.20).

- Train/test preprocessing mismatch: the training pipeline resized images to 286 pixels and cropped a 128×128 patch (roughly the central-area scale of a zoomed-in face), whereas test images show the whole face resized to 128×128. The paper's scheme crops 256 out of 286, which keeps scale nearly constant; with a 128 crop the scale differs by about 2×. This likely contributes to the artefacts in Figure 6, especially for sketch → face, and fixing it (resize to about 143, crop 128) is a concrete improvement.
- Quality is limited by the reduced setting: 500 images per domain out of 22,334, 128×128 resolution, 6 residual blocks and 20 epochs. The original paper trains for 200 epochs. More data and epochs would be the most effective improvement.
- Hyperparameters: lr 2e-4 and β1 = 0.5 gave stable training with no divergence. Batch size 4 (instead of the paper's 1) was chosen to speed up each epoch on a GPU, at the cost of noisier instance-norm statistics per image being averaged; training remained stable.

2.5 User interface

A Flask web application (app/app.py) loads the two trained generators and exposes a single page where the user can upload an image or capture one from the webcam. The server automatically decides whether the input is a sketch or a real face and runs the corresponding generator:

- Detection heuristic: a sketch is nearly colourless and mostly white, so the image is converted to HSV and treated as a sketch if its mean saturation is below 0.10 and its median brightness above 0.65; otherwise it is treated as a real face.
- The page shows the input and the result side by side with a heading naming the detected type. A manual override (Auto-detect / Face / Sketch) is provided for ambiguous images.
- Endpoints: GET / serves the page; POST /convert accepts the image and returns the translated image as a PNG data URL plus the detected type. Errors (missing weights, unreadable image) are returned as JSON messages.

Figure 8 shows a live camera capture converted in auto-detect mode: the input was classified as a real face and translated into a sketch. Figure 9 shows the same capture with the input type manually forced to "Sketch": the sketch → face generator is then applied to a photograph and returns a restyled colour image. This confirms that the manual override works, and shows why correct detection of the input type matters.

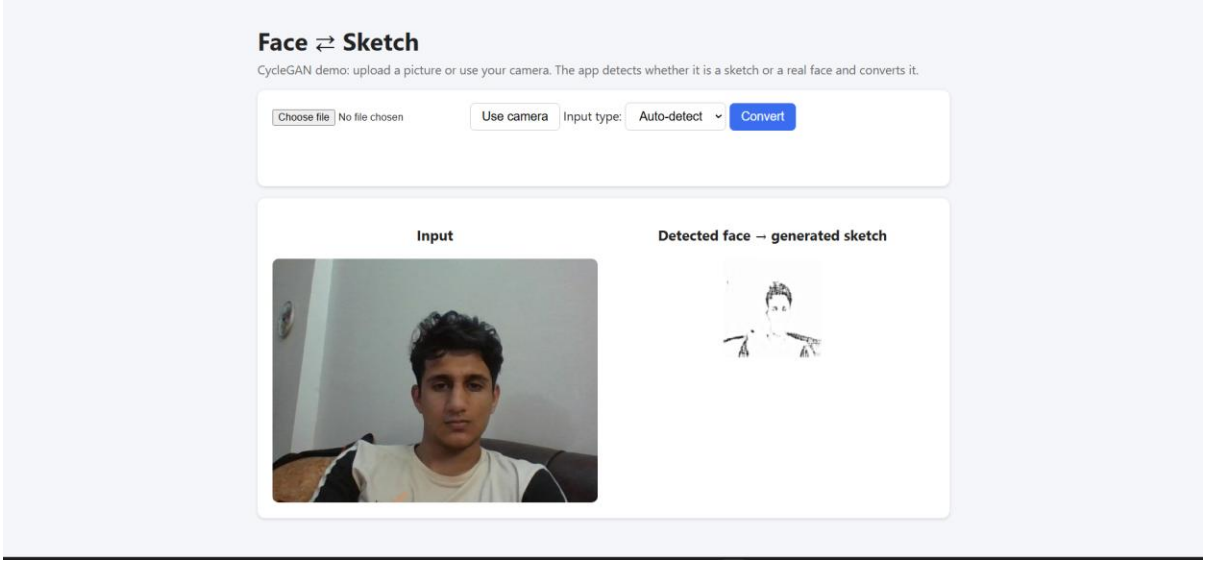


Figure 8: Web UI, camera input, auto-detect: face → generated sketch.

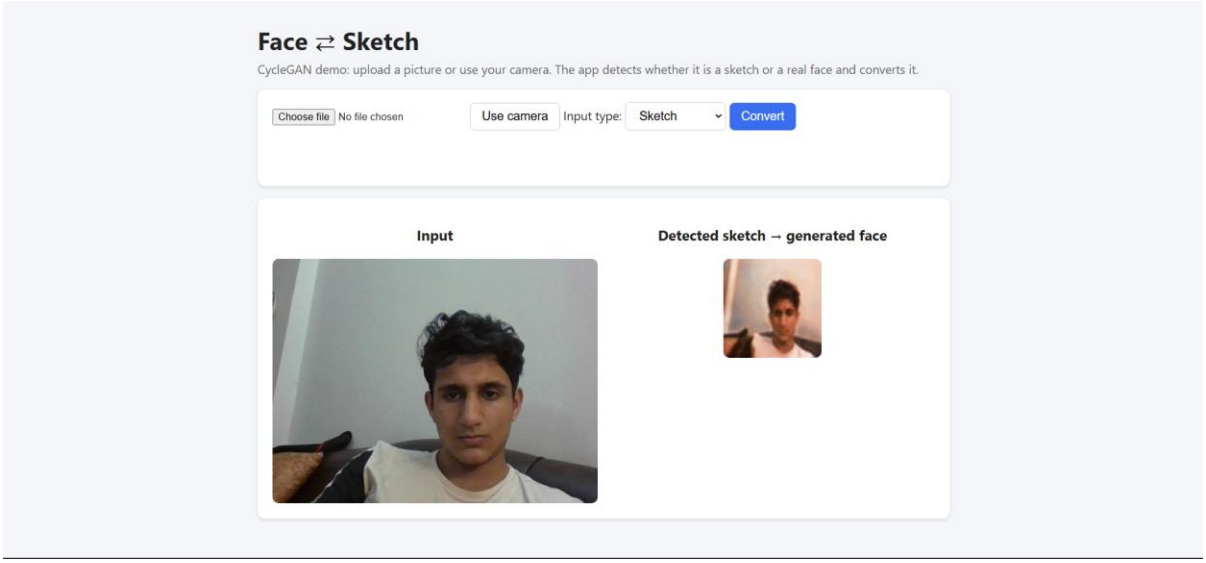


Figure 9: Web UI, input type manually set to Sketch: sketch → face generator applied.

Limitations of the UI: the colour-based detector can misclassify heavily desaturated (black-and-white) photographs as sketches and tinted scans of sketches as faces; the manual override handles those cases. Inference runs on CPU or GPU on 128×128 inputs and takes a fraction of a second per image.

Conclusion

A convolutional denoising autoencoder reconstructs clean BSDS300 images from Gaussian and salt-and-pepper noise, with the largest improvement over the noisy input at high noise levels and for impulse noise, and with performance limited mainly by bottleneck capacity at small sizes. A CycleGAN trained end-to-end on an unpaired subset of the Person Face Sketches dataset learns both face → sketch and sketch → face mappings, with checkpoint-and-resume training, and was deployed in a Flask application

that automatically detects the input type. Results for the sketch → face direction are modest, and training on the full dataset for more epochs at higher resolution is the clear next step.

References

- Zhu, J.-Y., Park, T., Isola, P., Efros, A. A. Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. ICCV 2017.
- Martin, D., Fowlkes, C., Tal, D., Malik, J. A Database of Human Segmented Natural Images and its Application to Evaluating Segmentation Algorithms. ICCV 2001 (BSDS).
- Person Face Sketches dataset, Kaggle: kaggle.com/datasets/almightyj/person-face-sketches.